

Code Crash Arena (실시간 1:1 알고리즘 대결)

2025.11 ~ 2025.12
(1인 개발)

소개

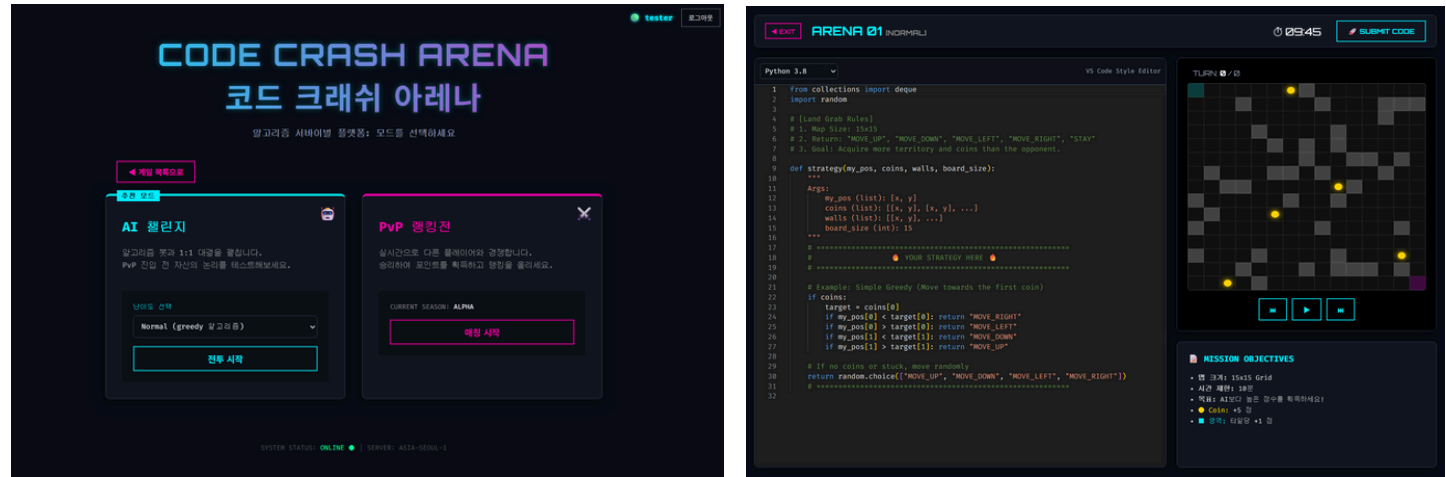
Docker 컨테이너 기반의 안전한 코드 실행 환경과 Redis를 활용한 매칭 시스템을 갖춘 실시간 대결 서비스입니다.

기술 스택:

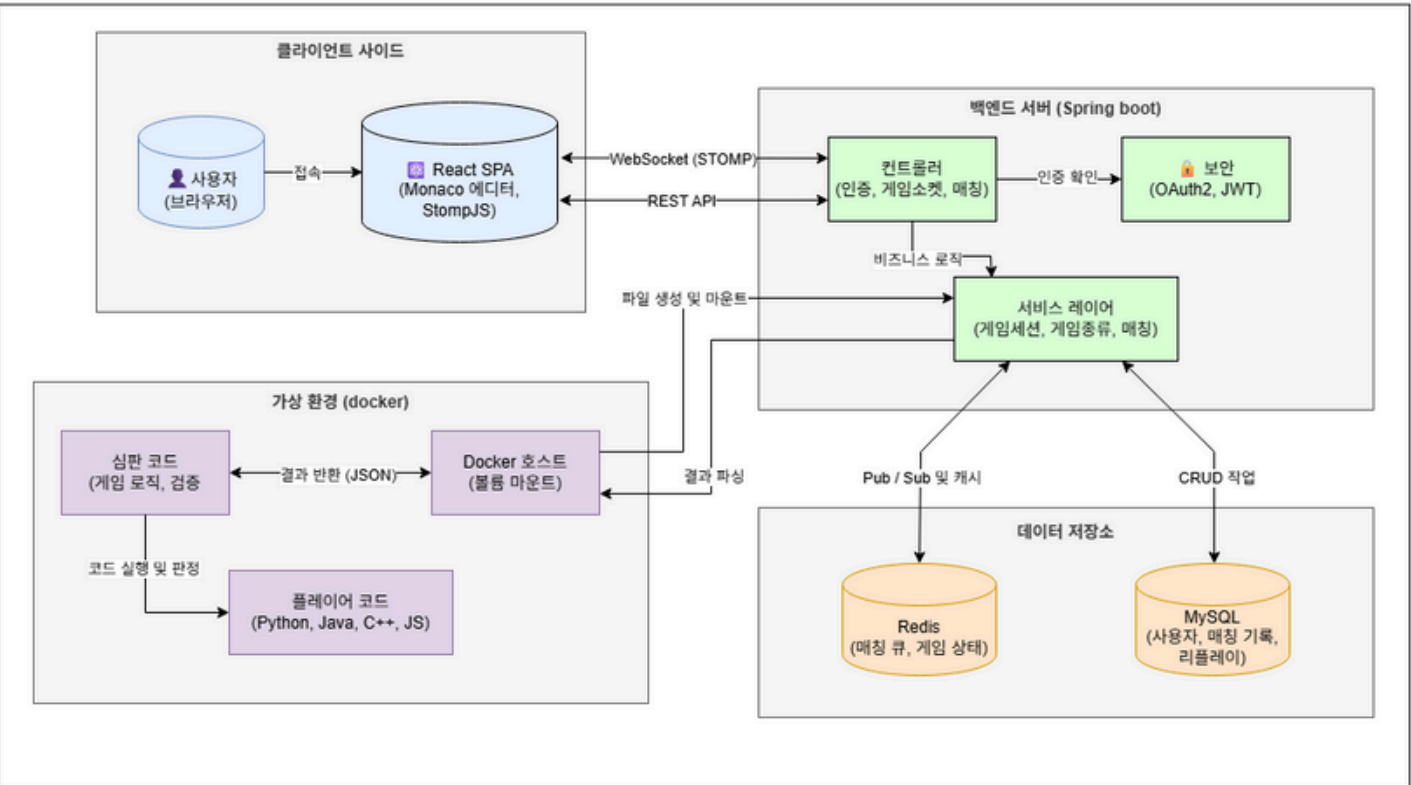
- **Frontend:** React
- **Backend:** Java 17, Spring Boot 3, JPA
- **Infra/DB:** MySQL, Redis, Docker
- **Protocol:** WebSocket, HTTP

주요 기능

- 여러 언어로(Java, Python, C++ 등) 코드 제출 및 채점 기능
- 실시간 1:1 매칭 및 게임 진행 상태 동기화
- 결과에 따른 승패 판정 및 전적 기록, 비정상 종료 시 기권 처리
- JWT를 이용한 로그인 (일반, 게스트, OAuth2)



시스템 아키텍처



[Issue] 대규모 요청 시 매칭 병목 및 Race Condition 발생 #1

Problem DB 기반 매칭 시 트랜잭션 락으로 인한 성능 저하 및 동시 요청 시 중복 매칭 가능성

Solution

- **In-Memory 전환**

- 기존 RDB의 Table 조회 방식 대신, I/O 속도가 빠른 Redis의 정렬된 집합(ZSET) 자료구조를 대기열로 사용
- 유저 ID를 멤버로, 대기 시작 시각을 score로 저장해 가장 오래 기다린 유저부터 매칭하도록 구성

- **Atomic Operation 활용**

- Redis는 단일 스레드 기반으로 각 명령을 차례로 처리하여, 개별 명령 수준의 원자성을 보장하는 점을 이용
- ZADD로 대기열에 입장시키고, 매칭 서버가 ZPOPMIN으로 가장 오래된 유저를 하나씩 꺼내는 구조를 채택

Result 동시성 이슈 해결 및 매칭 로직 처리 속도 대폭 개선

[Issue] 사용자 코드 실행 시 보안 위험 #2

Problem 무한 루프나 시스템 명령어가 포함된 코드가 서버 전체에 영향을 줄 위험이 존재

Solution

- **Docker 컨테이너 격리**

- 미리 여러 언어의 실행이 가능한 docker 이미지 빌드
- 코드를 복사하여 호스트 OS와 격리된 Docker 컨테이너에서 코드를 실행

Result 메인 서버의 안정성을 보장하면서 다중 언어 컴파일/실행 환경 구축

[Issue] WebSocket 세션 관리 및 네트워크 불안정 대응 #3

Problem React SPA의 로비에서 게임 화면으로 전환되거나 유저의 네트워크가 일시적으로 끊길 때 소켓 연결이 해제되어 세션이 유실되거나 좀비 세션이 남는 문제 발생

Solution

- **Redis Scope 분리**

- 유저의 상태 정보를 Redis 내에서 Lobby Scope와 Game Scope로 분리하여 저장
- 화면 이동으로 소켓이 재연결되더라도 Redis에 저장된 Game Scope 정보를 조회하여 기존 게임 방으로 재진입

- **비정상 종료 기권 처리**

- 설정된 시간 내에 세션이 복구되지 않으면 Redis에서 해당 유저의 상태를 만료시키고 상대방에게 승리(상대방 기권) 메시지를 전송하는 자동화 로직 구현

Result 네트워크 끊김 상황에서도 적절한 사용자 경험(UX) 제공 및 승패 처리

위드유 (AI를 활용한 주식 모의투자 플랫폼)

2025.03 ~ 2025.06
(4인 개발)

소개

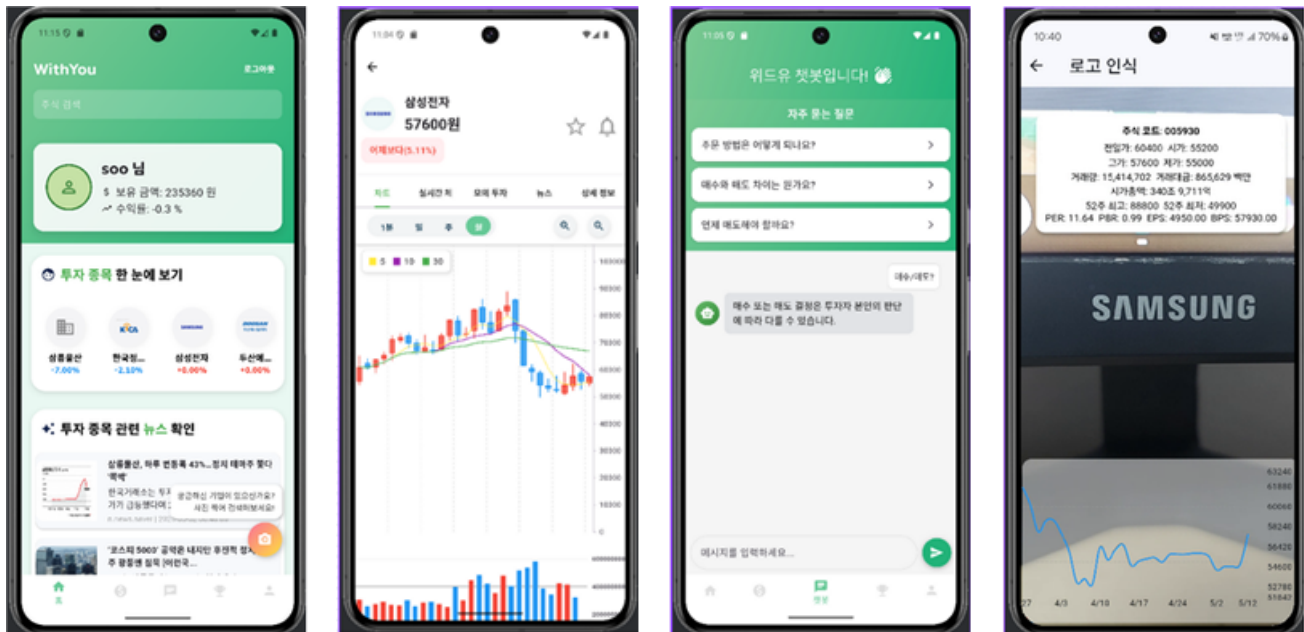
실제 주식 시세 API와 AI 챗봇, 로고 인식, 예약 거래 시스템으로 투자 전략을 실험할 수 있는 AI를 활용한 모의투자 서비스입니다.

기술 스택:

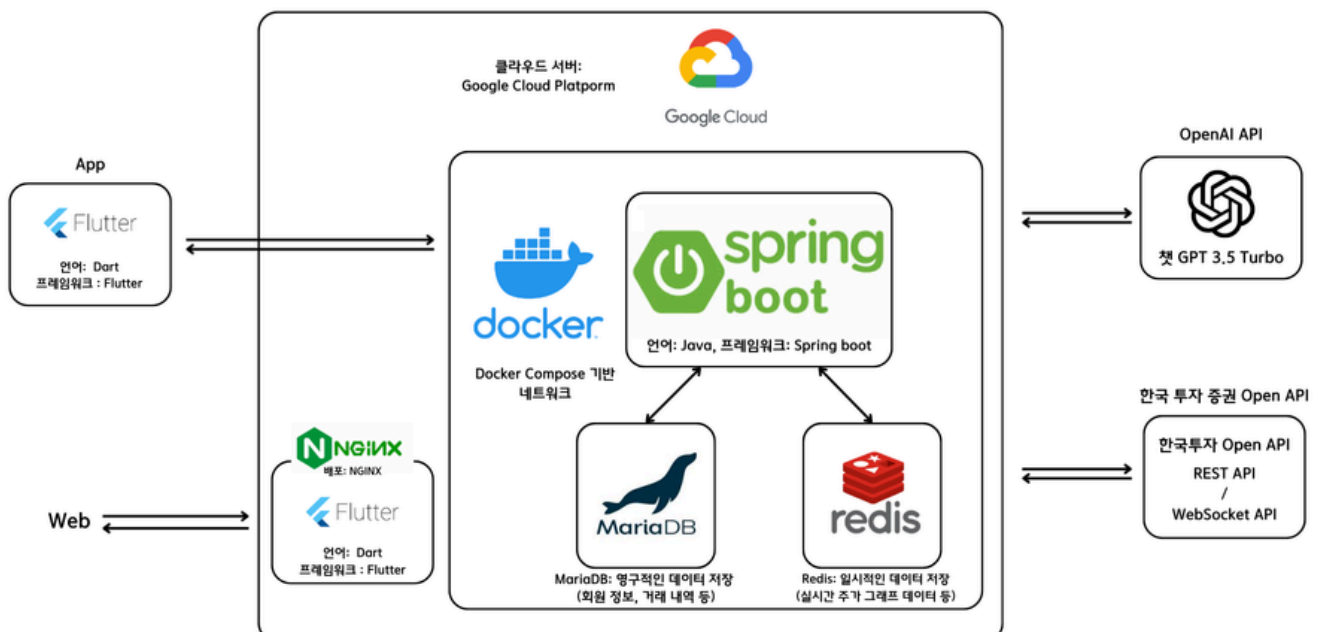
- **Frontend:** Flutter (Dart)
- **Backend:** Java 17, Spring Boot 3, JPA
- **Infra/DB:** MariaDB, Docker, Redis
- **Protocol:** HTTPS, GCP

주요 기능

- 실제 데이터를 기반으로 한 가상 거래
- 챗봇과 내부 API를 연계한 실제 데이터 응답 (주식 시세, 순위 등)
- 실시간 뉴스 요약, 탐색 기능으로 투자 정보 습득
- 카메라 기반 로고 인식으로 기업 탐색



시스템 아키텍처



[Issue] 주식 API 호출 시 간헐적 요청 횟수 초과 발생 #1

Problem 주식 API의 제공 횟수는 초당 20회 였지만 초당 4~5회에도 간헐적으로 발생하는 요청회수 초과 에러로 인해 데이터가 갱신되지 않는 문제 발생

Solution

- 단일 진입점 구성 및 딜레이 추가

- 모든 주식 API 요청을 관리하는 구조로 변경 후 Rate Limit으로 전체 요청이 초당 5회를 넘지 않도록 설정
- (일부 개선은 있었으나 이것만으로 해결되지 않음)

- 재요청 로직 및 이중 방어

- synchronized 블록과 마지막 요청 시각 기반 지연 처리로 멀티스레드 환경에서도 과다 요청 방지
- 그럼에도 실패하는 요청은 요청 큐에 다시 추가하여 재요청 수행

Result 요청 실패 최소화 및 트래픽 집중 상황에서도 안정적인 주식 데이터 수집

[Issue] 챗봇의 내부 API 호출 방식 설계 #2

Problem 사용자가 챗봇에게 주가나 특정 종목 순위 등과 같이 LLM의 사전 학습 지식만으로는 답변할 수 없는 실시간 데이터 기반 질문을 요청하는 경우가 발생

Solution

- 답변 포맷의 JSON 기반 표준화

- LLM이 행동 목록을 JSON 형태로 응답하도록 제한, 단순 질의 응답과 API 호출이 필요한 요청을 분리
- LLM이 사용자 질문을 해석하여 { "intent": "주식_현재가_조회", "parameter": {...} }와 같이 답변하도록 유도

- 단계적 처리 파이프라인 구성

- LLM 응답 → intent 판단(필요 시 내부 API 호출) → 반환된 실시간 데이터로 사용자 친화적인 최종 답변 생성

- 한계 인식 및 대안

- LangChain의 ReAct 패턴과 tool 기반 호출 방식이 보다 효율적인 해결책
- 개발 당시에는 LangChain 기술을 접하지 못해 보다 비효율적인 과정으로 구현됨

Result LLM과 내부 API를 결합해 사용자는 챗봇을 통해 사용자는 실시간 정보를 빠르게 확인

[Issue] 대규모 거래 요청 시 DB 병목 및 처리 지연 발생 #3

Problem DB에 사용자 정보, 거래 기록, 종목 정보를 모두 저장할 경우, 실시간 시세 조회 및 예약 거래 요청이 증가할수록 반복 조회로 인한 DB 부하 증가와 응답 지연 발생

Solution

- 데이터 역할 분리

- 사용자 정보, 거래 내역, 종목 메타데이터는 RDB에 저장
- 실시간 시세, 반복 조회 데이터는 Redis에 캐시하여 처리

- Redis 캐시 적용

- 종목 시세 조회 시 Redis 우선 조회 적용
- TTL 기반 갱신으로 데이터 최신성 유지

Result DB 접근 빈도 감소로 병목 현상 해소 및 실시간 조회·예약 거래 처리 성능 안정화

한컷 여행 (여행 경로 추천 및 공유 서비스)

2025.07 ~ 2025.09
(3인 개발)

소개

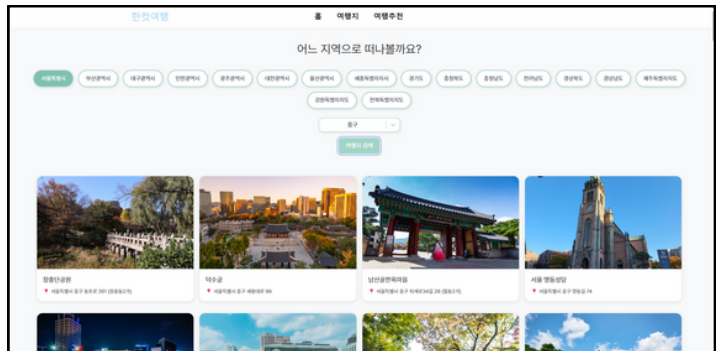
한국관광공사 API를 활용한 맞춤형 여행경로를 생성, 수정, 공유할 수 있는 서비스입니다.

기술 스택:

- **Frontend:** React
- **Backend:** Java 17, Spring Boot 3, JPA
- **Infra/DB:** MariaDB, Docker
- **Protocol:** HTTPS, AWS, Route53

주요 기능

- 혼잡도 별로 지역의 주요 관광지를 앨범 형식으로 추천
- 선택한 지역의 관광지를 한눈에 확인할 수 있는 기능
- 관광지 정보, 연관 관광지, 맛집, 숙소 정보를 확인하고 여행 코스를 편집·공유
- 키워드와 관련된 관광지들을 탐색



[Issue] API별 상이한 비정형 데이터의 통합 모델링 #1

Problem 한국관광공사 Open API의 다른 API간 지역별 코드명, 데이터 변수명이 일관되지 않아 데이터 통합 및 재사용에 어려움이 발생

Solution

- **통합 모델링 구성**
 - 의미는 동일하지만 다른 필드명을 가진 데이터를 하나의 통합 엔티티로 매핑
 - API별로 구조가 상이한 비정형 데이터는 메타데이터 테이블로 분리하여 정규화
 - 확장 가능한 구조로 설계하여 신규 API 추가 시 최소한의 수정으로 대응 가능

Result 통합된 데이터를 기반으로 내부 API에서 재사용 가능하도록 개선됨

[Issue] API 간 상이한 관광지 식별 체계로 인한 데이터 연계 문제 해결 #2

Problem 방문 순위 API, 관광지 정보 API 등에서 사용하는 관광지 식별 코드 체계가 서로 달라 동일 관광지에 대한 데이터 재사용 및 연계 활용에 어려움 발생

Solution

- **매칭 알고리즘 및 검증 로직**
 - 각 API 호출 후 저장된 데이터를 기준으로 관광지 명칭을 활용해 1차 매칭 수행
 - 매칭 후보 데이터에 대해 위도·경도 기반 거리 계산 알고리즘을 적용하여 일정 거리 이내 여부를 검증
 - 명칭 및 좌표 검증을 모두 통과한 데이터만 동일 관광지로 판단하여 매칭 정보 테이블에 저장

Result 연계된 데이터를 추천 로직에 반영하여 사용자에게 더 신뢰도 높은 여행 경로 추천 제공